Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

I Prova di Ingegneria del Software

27 Aprile 2004

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h30m.
6. Punteggio totale a disposizione: 13/30.

Esercizio 1 (punti 3)

Specificare la seguente astrazione procedurale *getInterval*, tramite pre e post condizioni:

- Sia dato un array *times*, che si ipotizza non nullo e che contiene una sequenza di valori float ordinati per valori strettamente crescenti (che rappresentano punti dell'asse dei tempi); sia dato inoltre un valore *timePoint* di tipo float. Fornire come risultato della chiamata un oggetto della seguente classe *Interval*, corrispondente a un intervallo temporale e avente come estremi due punti contigui di *times,* tale che *timePoint* sia maggiore o uguale all'estremo inferiore e minore dell'estremo superiore. Se il valore di *timePoint* è minore del primo valore di *times* o maggiore o uguale dell'ultimo viene lanciata l'eccezione OutOfBoundsException.

Si utilizzi la seguente breve specifica della classe Interval.

```
class Interval {//OVERVIEW...
    //AF(c) = [c.min(), c.max() )
    …costruttori e metodi varii…
    float min()
    //EFFECTS: restituisce l'estremo inferiore di this
    float max()
    //EFFECTS: restituisce l'estremo superiore di this
```

public static Interval getInterval (float[] times, float timePoint) .....
.....

public static Interval getInterval (float[] times, float timePoint) throws OutOfBoundsException
//REQUIRES: times<>null && times.length>1 &&
//forall i, 0<=i<times.length-1, times[i]< times[i+1]
//EFFECTS: if (timePoint<times[0] || timePoint>=times[times.length-1]) throw
//OutOfBoundsException
//else return Interval x tale che exists int i: times[i]=x.min() <=timePoint < x.max()=times[i+1];

- Dare una versione della specifica in cui nel caso il *times* sia nullo viene generata un'opportuna eccezione.

public static Interval getInterval (float[] times, float timePoint) .....

public static Interval getInterval (float[] times, float timePoint) throws OutOfBoundsException, *NullPointerException*

poi si elimina dal REQUIRES la condizione times<>null e si aggiunge all'inizio della clausola EFFECTS:
if (times ==null) throw NullPointerException

- Quali sono le differenze principali tra l'assumere che la condizione di vettore non nullo sia una precondizione piuttosto che la sua violazione generi un'eccezione?

Nel primo caso, si ipotizza che sia parte del "contratto" fra chiamante e chiamato che il chiamante rispetti la precondizione. In caso di mancato rispetto, il chiamato è "autorizzato" a qualunque tipo di comportamento. Non deve quindi mai accadere che il chiamante non rispetti la precondizione.
Se la violazione genera un'eccezione, invece, si ipotizza che il chiamante possa evitare di verificare la precondizione (e quindi anche non rispettarla), ma che sia in tal caso capace di gestire l'eccezione corrispondente alla violazione, dichiarata nella segnatura del metodo.

Esercizio 2 (punti 5)

Consideriamo la seguente specifica di un tipo di dato astratto modificabile (mutabile) che rappresenta una coda di messaggi con priorità. Ogni messaggio ha associata una priorità, che è un valore nell'intervallo 0-9 (0 = bassa, 9 = alta). Le operazioni possibili sono l'inserimento di un certo messaggio con una certa priorita` nella coda (*insert*) e l'estrazione (*extract)* dalla coda del messaggio da più tempo inserito fra quelli a priorità più alta presenti nella coda. Extract restituisce tale messaggio e lo elimina dalla coda. Un'operazione *length* restituisce la lunghezza della coda (numero di messaggi presenti in coda) e un'altra operazione *merge* riceve come parametro una coda che deve essere fusa con la coda corrente, facendo in modo che nella nuova coda corrente tutti i messaggi contenuti nella coda-parametro siano posti, a pari prorità, "dietro" a quelli della coda corrente.

1. Si completi la seguente specifica della classe CodaPriorità (supponendo nota la classe Messaggio), a partire dalla clausola OVERVIEW proposta, specificando per ciascun metodo, le clausole REQUIRES, MODIFIES e EFFECTS (se necessarie). Si specifichi anche quali operazioni siano costruttori, mutators e observers. Per ogni metodo si completi il prototipo, segnalando eventuali eccezioni.

(NB: ovviamente l'ammontare di righe vuote o di puntini non è correlato con la lunghezza della soluzione…)

public class CodaConPriorita {
/*OVERVIEW: Una coda con priorità è una sequenza di coppie del tipo (x,n) dove x è un Messaggio e n la sua priorità, 0<=n<=9.
La sequenza è composta da una successione di 10 "segmenti" (anche vuoti), ognuno dei quali contiene messaggi con la stessa priorità, disposti, all'interno del segmento, in ordine cronologico di inserzione nella coda (a sinistra, cioè con posizioni di indice più basso, i messaggi inseriti prima); il primo segmento contiene i messaggi a priorità 0, il secondo quelli a priorità 1, e così via.
Esempio: (m1, 3) (m4, 3) (m2,4) (m3,4) (m6,4) (m5,6) */

public CodaConPriorita () { **COSTRUTTORE**
//EFFECTS: costruisce una coda vuota (senza elementi)
}
//Metodi:

public void insert(Messaggio m, int priorita) **throws WrongPriorityException**…… { **MUTATOR**
//EFFECTS: if (0<= priorita <=9) inserisce (m,priorità) alla destra di tutti gli elementi con minore o uguale priorità, e a sinistra di tutti quelli con priorità maggiore ……
//else throw WrongPriorityException ……
//MODIFIES: this……
}
public Messaggio extract() **throws EmptyException**……{ **MUTATOR+OBSERVER**
//EFFECTS: if (this non è vuota), then, se max è la massima priorità presente nella coda, restituisce il Messaggio m corrispondente alla coppia (m,max) più a sinistra nella coda, al contempo eliminandolo dalla coda.……
//else throw EmptyException ……
//MODIFIES: this……
}

public void merge(CodaConPriorita c) **throw NullPointerException** ……{ **MUTATOR**
……
//EFFECTS: if (c == null) then throw NullPointerException
// this_post è una sequenza in cui sono presenti nello stesso ordine relativo, tutti gli elementi di //this_pre, e tutti gli elementi di c di prorità i compaiono, nell'ordine originale di c subito dopo tutti gli //elementi di this a priorità i. ……
//MODIFIES: this……
}

public int length()……{ **observer**
……
//EFFECTS: return il numero di elementi di this……
……
}

}

NB: La insert potrebbe anche verificare se il Messaggio m è vuoto, e in tal caso non inserire l'elemento ma lanciare un'opportuna eccezione

2. Si consideri un'implementazione per la quale la rep scelta sia un array di 10 riferimenti a vector che rappresentano i messaggi accodati per le diverse priorità, in modo che il messaggio ricevuto più recentemente per una certa priorita` sia il LastElement del vector relativo. Inoltre un attributo privato intero maxPriorita memorizza la massima priorità presente in quel momento nella coda. Quando la coda è vuota maxPriorita valga –1. Data la rep, si definiscano anche l'AF e l'RI. Completare l'implementazione seguente, indicando brevemente, per ciascun metodo, (a) perché il rep invariant viene effettivamente mantenuto e (b) perché la specifica fornita viene rispettata.

//rep:
private Vector[] coda;
private int maxPriorita;…
//AF(c):
// una sequenza C = C0…C9 formata dalla concatenazione di 10 segmenti Ci, ognuno dei quali è vuoto se e solo se coda[i] è vuota, altrimenti Ci = (coda[i].firstElement(),i)…(coda[i].lastElement(), i)
//RI(c) =
// coda è un array di 10 elementi di tipo Vector (!= null) &&
//tutti gli elementi nei Vector della coda sono di tipo Messaggio &&
se tutti i Vector sono vuoti, maxPriorita == -1, altrimenti maxPriorita è il massimo indice i per cui il Vector coda[i] contiene almeno un elemento ……..

```
public  CodaConPriorita ()…….. {
      coda=new Vector[10];
      for (int i =0; i<10; ++i)
            coda[i]=new Vector();
      maxPriorita = -1;
}
```

Mantiene RI perché:
Per costruzione, maxPriorita è -1……..
……
Rispetta specifica perché
Per la AF, se tutti i Vector coda[i] sono vuoti l'insieme di elementi è pure vuoto. ……..

```
public void  insert(Messaggio m, int priorita) throws WrongPriorityException …….. {
    if (0<= priorita && priorita <=9) {
      coda[priorita].add(m);
      if (maxPriorita < priorita) maxPriorita = priorita;
    }
    else throw new WrongPriorityException();…….
}
```

Mantiene RI perché:
……
Inserisce un elemento di tipo Messaggio; aggiorna correttamente maxPriorita nel caso il nuovo messaggio abbia priorità maggiore di tutti quelli presenti ……..

Rispetta specifica perché
Aggiunge m alla destra (come LastElement) di tutti gli elementi con la stessa priorità. La tesi segue perche' AF mantiene l'ordinamento degli elementi in coda con la stessa priorità.

```
public Messaggio extract() throws EmptyException …….{
      if (maxPriorita == -1) throw new EmtpyException();……
      Messaggio m = (Messaggio) coda[maxPriorita].get(0);
      coda[maxPriorita].remove(0);
      while (maxPriorita >= 0 && coda[maxPriorita].size() == 0)
            maxPriorita--;
      return m;
}
```

Mantiene RI perché:
aggiorna correttamente maxPriorita nel caso in cui non rimanga alcun messaggio con la stessa priorità` di quello estratto
Rispetta specifica perché
trova l'i massimo corrispondente al Vector non vuoto e poi ne prende il primo elemento: per AF, questo è l'elemento a priorità massima e piu' a sinistra.

```
public void merge(CodaConPriorita c) throw NullPointerException …… {
      for ( int i =10; i>=0; --i)
            for (int j=0; j<c.coda[i].size(); j++)
                  this.insert(c.coda[i].get(j), i);
}
```

Mantiene RI perché:
Non modifica struttura; non aggiunge ne' toglie Vector; tutti gli elementi aggiunti sono di tipo Messaggio (perche' RI vale per c). L'inserimento mantiene RI (perche' aggiorna maxPriorita)
……
Rispetta specifica perché
Il metodo inserisce tutti gli elementi di c nella corrispondente coda, rispettandone la sequenza di inserimento e mettendoli dopo tutti quelli di this. Poiché AF rispetta l'ordine, la specifica è verificata.

```
public int  length()……..{
    int len =0;
    for (int i = 0;i<=maxPriorita; ++i) len = len + coda[i].size();
    return len;……
}
```

Mantiene RI perché:
è un observer…….
……
Rispetta specifica perché
Restituisce la somma del numero di elementi in ogni coda: per la AF tutti e soli questi elementi sono in coda. ……..

3. Si fornisca un'implementazione di AF ed RI, in modo che sia utile ai fini del debugging del programma.

```
public String toString() {
      String s = "";
      for ( int i = 0; i < 10; i++)
                  for (int j=0; j< coda[i].size(); j++)
                        s = s + "(" + this.coda[i].get(j).toString() + ", " + i  + ")";
      return s;
}

public Boolean repOK()  {
      boolean vuota =true;
      if (coda.length!= 10) return false;
      for ( int i = 9; i >= 0; --i)
            if (coda[i].size()>0) {
                  vuota = false;
                  if (maxPriorita <i) return false;
            for (int j=0; j<coda[i].size(); j++)
                  if (!coda[i].get(j) instanceof Messaggio) return false;
            }
      if (vuota) return maxPriorita==-1;
      return true;
}
```

Esercizio 3 (punti 2)
Si consideri il seguente frammento incompleto di programma Java:

```
class MammaMia …{…};
class Mortacci …{…};
class CheSo …{
      void f (…) throws MammaMia, Mortacci {
            …
            throw new MammaMia();
            …
      }
      void g(…) throws MammaMia, Mortacci {
            …
            try {
                  …
                  f( );
            } catch (MammaMia m) { System.out.println('1');
            } finally { throw new Mortacci( );}
}
```

Si ipotizzi che l'eccezione sollevata all'interno del metodo f() non venga gestita da f(), ma venga propagata.

Si consideri il seguente frammento di codice che invoca il metodo g:

```
      CheSo oggetto = new CheSo ( );
      try {
            oggetto.g(…);
            …
      } catch (MammaMia m) { System.out.println('2');
      } catch (Mortacci m) { System.out.println('3'); }
```

Supponendo che la chiamata di f() comporti il lancio dell'eccezione MammaMia, si indichi che cosa stampa il programma in seguito alla chiamata del metodo g(..) sulla variabile *oggetto*.

1
3

Esercizio 4 (punti 3)
Rispondere alle seguenti domande relative a Java:

(A) Si consideri la seguente classe:

```
class StranaCosa {
        static int i = 0;
        int j = 0;
        void boh( ) {
            i++;
            j++;
            System.out.println(i);
            System.out.println(j);

        }
}
```

Si supponga che altrove appaia il seguente frammento:

```
StranaCosa x = new StranaCosa( );
StranaCosa y = new StranaCosa( );
x.boh( );
y.boh( );
```

Che cosa viene stampato dal programma?
1
1
2
1

(B) Si suppongano date le classi X e Y, dove Y è erede di X. Si supponga che X possieda un metodo:
`void a() {System.out.println( “a di X”);}`
mentre la classe Y ridefinisca il metodo come segue:
`void a() {System.out.println( “a di Y”);}`
In presenza del seguente codice:

```
X x = new Y();
x.a();
```

si indichi qual è il tipo statico di x e quale il suo tipo dinamico, e che cosa viene stampato.
Tipo statico: X, tipo dinamico Y. “a di Y”

(C) Sono date le classi X, Y, W e Z e con le seguenti relazioni di ereditarietà:
W eredita da Z; Y e Z ereditano da X. Le variabili x,y,z,w sono dichiarate come X x; Y y; W w; Z z;
In base alle dichiarazioni, quali tra le seguenti assegnazioni sono legali in Java? (Scrivere SI o NO a fianco di ciascuna.)

1. y = x; **NO**
2. x = y; **SI**
3. z = x; **NO**
4. x = w; **SI**
5. w = x; **NO**
6. y = z; **NO**